

**Experiential Learning
Project Assessment-I
on
“QR Code Encoder / Decoder”
under the course
Python Programming Language (N-PCCCM304P)**

**Presented By:
DEVIKA PARSATWAR
[CM25015]**



**Under the Guidance of
Ms. Suchita Meshkar**

**Session:2026-27 (ODD)
Department of CSE(AI&ML)
S. B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT AND
RESEARCH,NAGPUR
An Autonomous Institute, Affiliated to RTMNU, Nagpur**

Contents

- Problem Statement
- Objectives
- Algorithm
- Flowchart / Pseudocode
- Understanding of Python Logic
- Graphical Representation and Working
- GitHub Account, Repository & Collaboration
- Individual Contribution
- Result / Expected Output
- Python Source Code

1. Problem Statement

- Sharing long text, URLs and contact information manually can be inconvenient.
- QR codes provide a quick and efficient way to store and share digital information.
- Creating QR codes manually requires specialized tools or applications.
- Users also need a way to **decode existing QR codes** and retrieve their stored information.
- The system should allow users to generate QR codes from different types of input data.
- Generated QR codes should be saved as image files for future use.
- The application should decode QR codes from existing images.
- It should also support decoding QR codes using a **webcam**.
- The project demonstrates the practical application of **Python image processing and data encoding**.

2. Objective

- To develop a QR Code Encoder and Decoder application using Python.
- To generate QR codes from text, URLs and other user-provided data.
- To save generated QR codes as image files.
- To decode information stored inside an existing QR code image.
- To decode QR codes using a webcam.
- To provide a simple and user-friendly interface.
- To understand Python libraries used for QR generation and image processing.
- To demonstrate the practical application of Python in real-world data encoding.

.

3. Algorithm

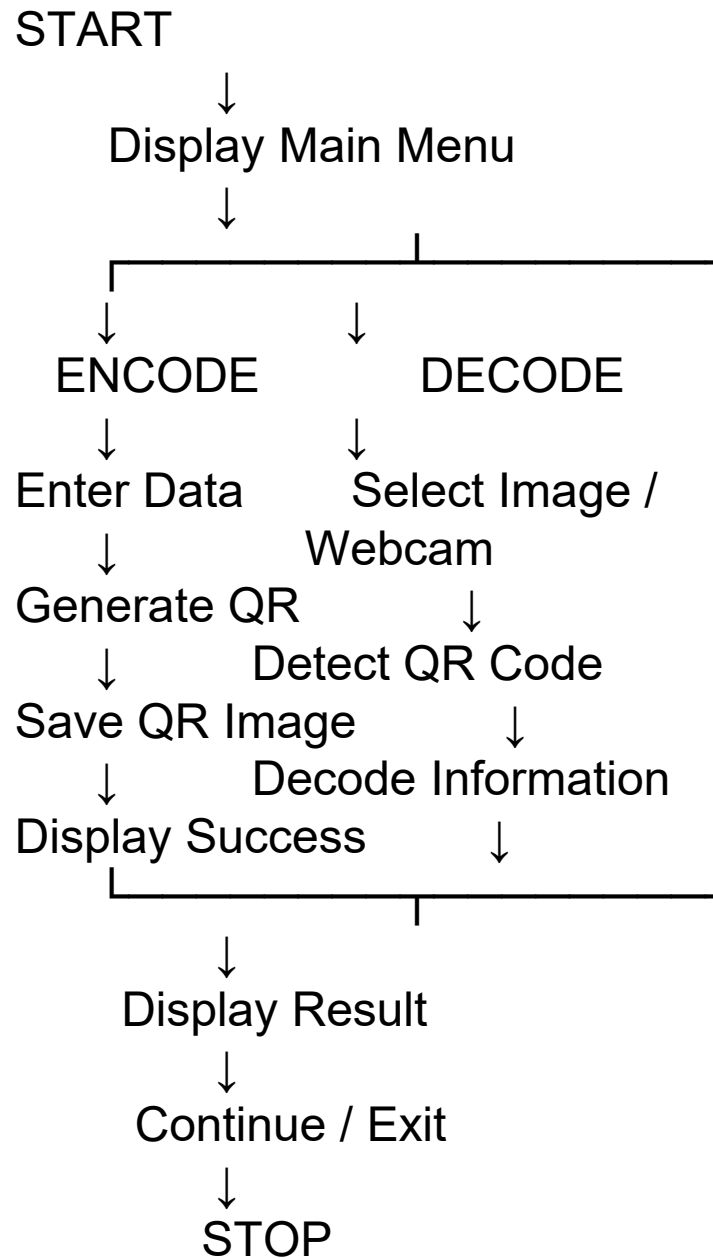
❖ QR Code Encoding:

1. Start the program.
2. Ask the user to enter text, URL or other information.
3. Receive the input data.
4. Create a QR code using the input data.
5. Convert the generated QR code into an image.
6. Save the QR code image with a suitable filename.
7. Display a success message.
8. Stop the encoding process.

❖ QR Code Decoding:

1. Start the decoder.
2. Select a QR code image or activate the webcam.
3. Read the image/frame.
4. Detect the QR code.
5. Decode the data stored inside the QR code.
6. Display the decoded information.
7. Stop the decoder.

4. Flowchart



5. Understanding of python logic

```
import qrcode
import cv2

# Generate QR
data = input("Enter data: ")
qrcode.make(data).save("QR.png")

# Decode QR
img = cv2.imread("QR.png")
detector = cv2.QRCodeDetector()
data, __, __ = detector.detectAndDecode(img)

print("Decoded Data:", data)
```

6. Graphical Representation and Working

6. Graphical Representation and Working

QR Code Encoder

- User enters text, URL or contact information.
- Python generates a QR code using the qrcode library.
- QR code is saved as an image file.
- The generated QR code can be shared with others.

QR Code Decoder

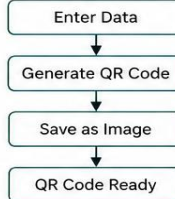
- User selects a QR code image or uses webcam.
- OpenCV detects and reads the QR code.
- The stored information is decoded and displayed.
- Works with both image files and live webcam feed.

Example (Python Logic)

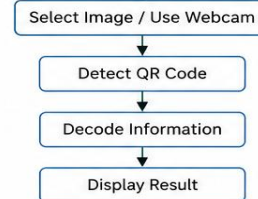
```
import qrcode, cv2
data = input("Enter data: ")
qrcode.make(data).save("QR.png")
img = cv2.imread("QR.png")
detector = cv2.QRCodeDetector()
data, _, _ = detector.detectAndDecode(img)
print("Decoded Data:", data)
```

Working Flow

Encoding Process



Decoding Process



QR Code Generated



(Generated from text/URL)

QR Code Decoded

Decoded Information

Data:
<https://www.example.com>
Status:
QR Code Detected Successfully!

(Decoded from image / webcam)

This project demonstrates the practical application of Python in QR code generation and decoding using image processing.

7. GitHub Account, Repository & Collaboration

- **GitHub Account:** Create and maintain a personal GitHub account.
- **Repository Creation:** Create a repository named **QR Code Encoder Decoder**.
- **Upload Code:** Upload Python source files and project documentation.
- **Meaningful Commits:** Use clear commit messages such as:
 - Added QR code generator
 - Added QR image decoder
 - Added webcam scanning
 - Updated documentation
- **Collaboration:** Team members can work using branches and pull requests.
- **Individual Contribution:** Each member can contribute code and documentation.
- **Track Changes:** GitHub keeps a history of project modifications.

8. Individual Contribution

- **Problem Analysis** — Studied the requirement for QR code encoding and decoding.
- **Algorithm Design** — Designed the encoder and decoder workflow.
- **Python Coding** — Implemented QR generation and decoding functionality.
- **Image Processing** — Worked with QR code images using Python libraries.
- **Webcam Integration** — Implemented QR detection through webcam.
- **Testing** — Tested QR generation and decoding with different inputs.
- **GitHub Contribution** — Uploaded source code and maintained project commits.
- **Documentation & PPT** — Prepared project documentation and presentation.

9. Result / Expected Output

```
=====
QR CODE ENCODER
=====
```

```
Enter data: https://google.com
```

```
QR Code generated successfully!
Saved as: generated_qr.png
```

Decoder:

```
=====
QR CODE DECODER
=====
```

```
QR Code detected!
```

```
Decoded Data:
https://google.com
```

Thank You

QR Code Encoder / Decoder